Podstawy Uczenia Maszynowego

Modele liniowe

Regresja (regression)

- **uczenie nadzorowane (supervised learning)**
- przewidujemy **wartość uporządkowaną**, zwykle liczbę rzeczywistą
- zmienna zależna / target
- minimalizujemy **błąd** - różnicę między rzeczywistością a predykcjami
- **zastosowania:**
  - click-through rate (CTR) reklam
  - rozpuszczalność molekuł
  - wartość spółek na giełdzie

Model liniowy (linear model)

- **model postaci:**

$$\hat{y} = w_0 1 + w_1 x_1 + ... + w_d x_d = x^T w$$

- uczymy się **wektora współczynników (wag)**
- **intercept (bias)** - sztuczna zmienna z samymi 1
- **założenia:**
  - 1 parametr per wymiar wejściowy
  - przewidywana wartość to **kombinacja liniowa** cech i wag

Modele liniowe

$$\hat{y} = w_0 1 + w_1 x_1 + ... + w_d x_d = x^T w$$

- **jak** wyznaczyć wagi?
- **zminimalizować** wybraną **funkcję kosztu**
- różne funkcje kosztu dają inne wyniki, algorytmy, koszt obliczeniowy etc.

- **uwaga:** często nazywa się je "modelami", np. "model regresji liniowej"
- tak naprawdę to algorytmy wyznaczania współczynników dla modelu liniowego

Modele liniowe

$$\hat{y} = w_0 1 + w_1 x_1 + \ldots + w_d x_d = x^T w$$

Nazwa	Funkcja kosztu	Algorytm (optymalizator)
Linear regression	MSE	SVD
Ridge reg.	MSE + L2	SVD i inne
LASSO reg.	MSE + L1	Coordinate Descent (CD)
ElasticNet reg.	MSE + L2 + L1	CD
Quantile reg., LAD	Pinball loss, MAE	Programowanie liniowe
Huber reg.	Huber loss	L-BFGS
Linear SVM	Hinge loss	Dual CD

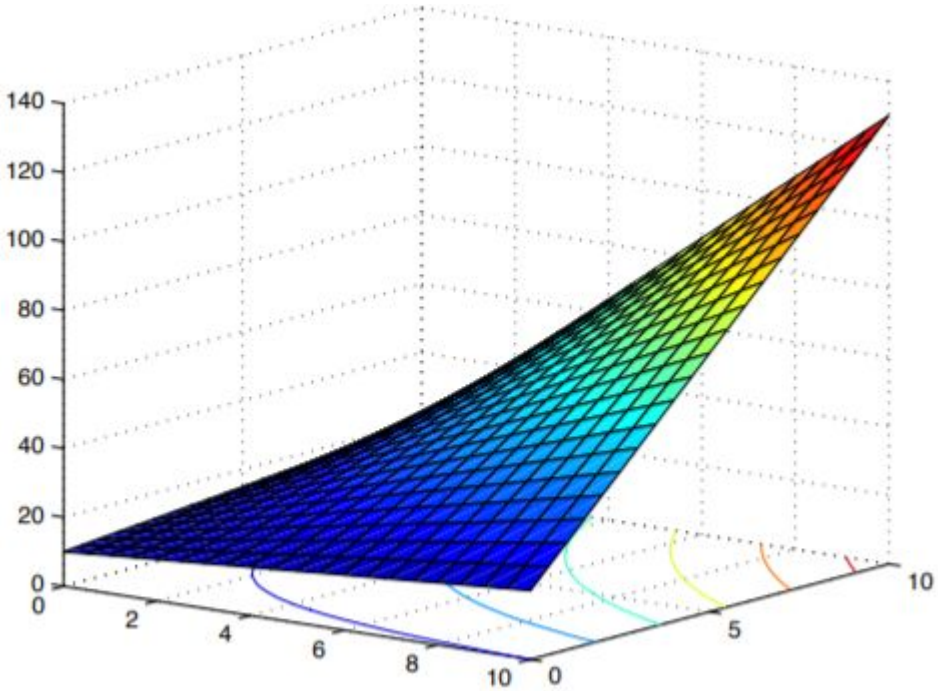
"Liniowość" modeli liniowych

Wbrew nazwie wartości przewidywane modelem liniowym **nie muszą** być linią prostą / płaszczyzną / hiperpłaszczyzną!

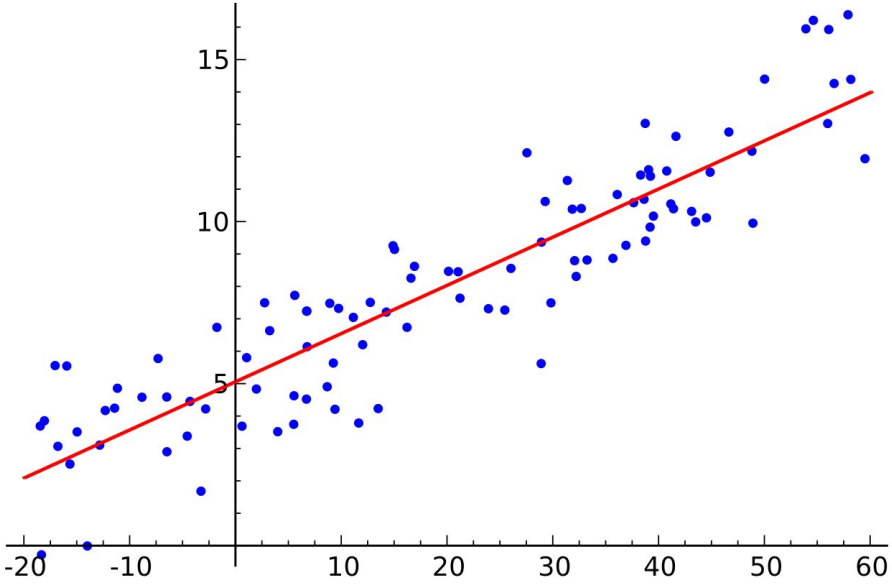
$$y = 10 + x_1 + x_2 + x_1 x_2 = w \cdot x$$

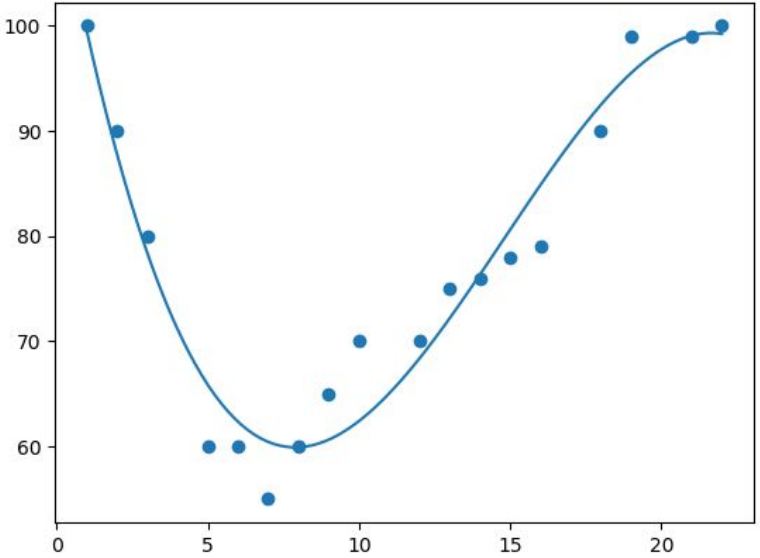
$$x = [1, x_1, x_2, x_1 x_2]$$

$$w = [10, 1, 1, 1]$$



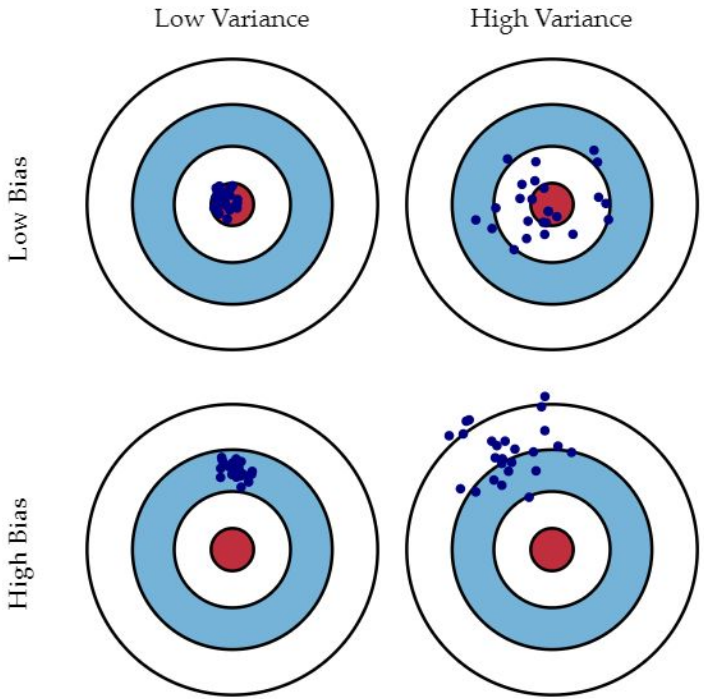
"Liniowość" modeli liniowych





Obciążenie i wariancja modeli liniowych

- **duże obciążenie (bias):**
  - mocne założenia, mają mało parametrów
  - można zmniejszyć dobrą inżynierią cech
- **niska wariancja (variance):**
  - jak jest dobrze - tego chcemy!
  - odporne na szum



Korelacje liniowe

- **założenie** modeli liniowych:
  - idealnie - zmienne są **ortogonalne**, zero korelacji liniowych
  - co najmniej - brak zmiennych **zależnych liniowo (multicollinearity)**
- w praktyce są jakieś korelacje
- z każdej **grupy** skorelowanych zmiennych bierzemy jedną, np. losowo, albo z największą wariancją
- jest to najprostsza metoda **selekcji cech**
- **regularyzacja** też pomaga

One-hot encoding

- modele liniowe wymagają **zmiennych numerycznych**
- one-hot encoding jest wygodny, bo zamienia zmienne kategoryczne na skalę [0, 1]
- **problem:** tradycyjny one-hot encoding daje K cech dla K kategorii, i są one liniowo zależne!
- tzw. **dummy variable trap**
- **rozwiązanie:** robimy K-1 kategorii, gdzie pierwsza kategoria jest implicite zakodowana jako zera

One-hot encoding - przykład

- poniższy przykład unika dummy variable trap

color
R
G
B
R
G

color_ R	color_ G
1	0
0	1
0	0
1	0
0	1

Normalizacja w modelach liniowych

- wiele modeli liniowych **wymaga** zmiennych o tej samej skali
- ważne dla szybkości optymalizacji, **wymagane** do regularyzacji i badania ważności cech
- **scaling (min-max normalization):** przeskalowanie zmiennych do zakresu [0,1]; szczególnie wygodne, gdy mamy zmienne po one-hot encodingu
- **standardization:** centrowanie i zamiana wariancji na jednostkową, daje tzw. $z = \frac{x - \mu}{\sigma}$ nagana do analizy statystycznej modelu

Ważność cech w modelach liniowych

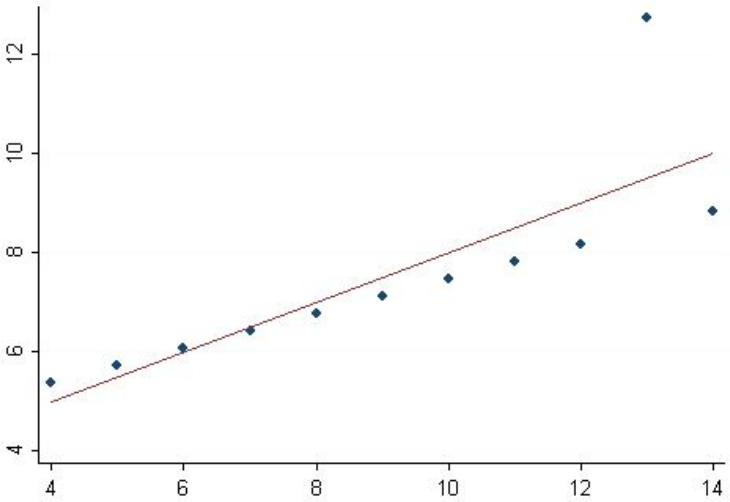
- wielkość (bezwzględna) wagi wyznacza **czułość** modelu na zmiany danej cechy
- duża waga cechy = model na niej mocno polega - miara **ważności cech**
- wymaga **znormalizowanych** cech, żeby wagi były tej samej skali
- **heurystyczna**, nie bierze pod uwagę interakcji między cechami, wielkości próbki etc.
- bierze pod uwagę **kierunek** zależności (pozytywny / negatywny wpływ)
- zależy od **jakości** modelu - możemy ufać, gdy mamy dobry model (mały błąd)

Regresja liniowa

Regresja liniowa

$$L(w) = MSE(w) = \frac{1}{n}(y_i - \hat{y}_i)^2 = \frac{1}{n}||y - Xw||^2$$

- **funkcja kosztu:** mean squared error (MSE)
- **algorytm:** Singular Value Decomposition (SVD)
- estymuje **średnią warunkową (conditional mean)**
- **zalety:** proste, szybkie, różniczkowalne, zamknięty wzór, łatwo regularyzować
- **wady:** często zbyt proste, podane na outliery



Regresja liniowa - zastosowania

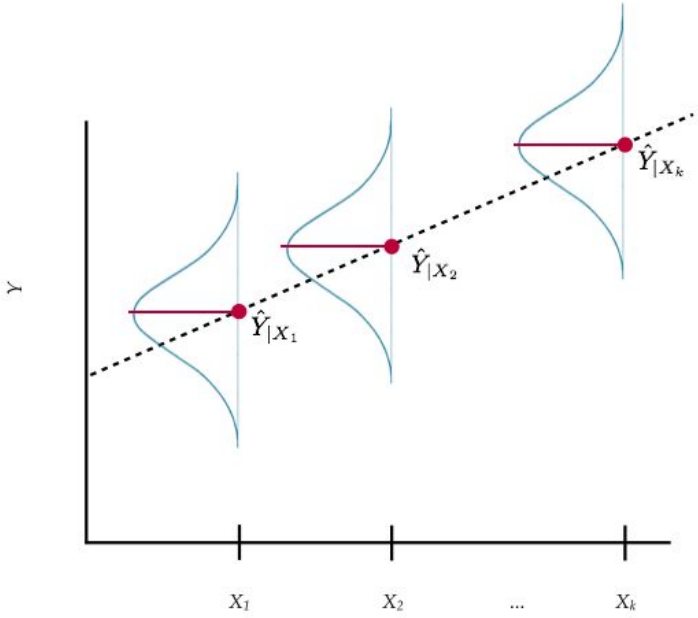
- **baseline** - najprostszy model, daje często niezłe wyniki
- **zastosowania real-time** - np. real time bidding (RTB)
- **ekonomia** - np. model CAPM, współczynnik beta
- **nauki przyrodnicze, medycyna** - analiza regresji, testy statystyczne etc.
- **przewidywanie szeregów czasowych** - podstawowa metoda oraz element wielu innych algorytmów, np. w ekonometrii

Regresja liniowa - średnia warunkowa

- prosta w regresji liniowej przechodzi przez **średnią** dla każdego zestawu cech *x*
- jest to **średnia warunkowa (conditional mean)**

$$\mathbb{E}(Y|X)$$

- **przykład:** *person that has a height of 170cm, weights on average 70kg*
- więcej: link 1, link 2, link 3



Rozwiązanie analityczne - SVD

- z danych treningowych mamy:

$$y = Xw \implies w = X^{-1}y$$

- **problem:** mamy n >> d (układ nadokreślony)
- X nie jest kwadratowe, nie da się odwrócić, więc **przybliżymy** tę odwrotność
- **Moore-Penrose pseudoinverse** to najlepsze przybliżenie odwrotności macierzy
- użycie tej pseudoodwrotności daje rozwiązanie **najmniejszych kwadratów (Ordinary Least Squares, OLS)**

Rozwiązanie analityczne - SVD

- pseudoodwrotność oblicza się z **Singular Value Decomposition (SVD):**

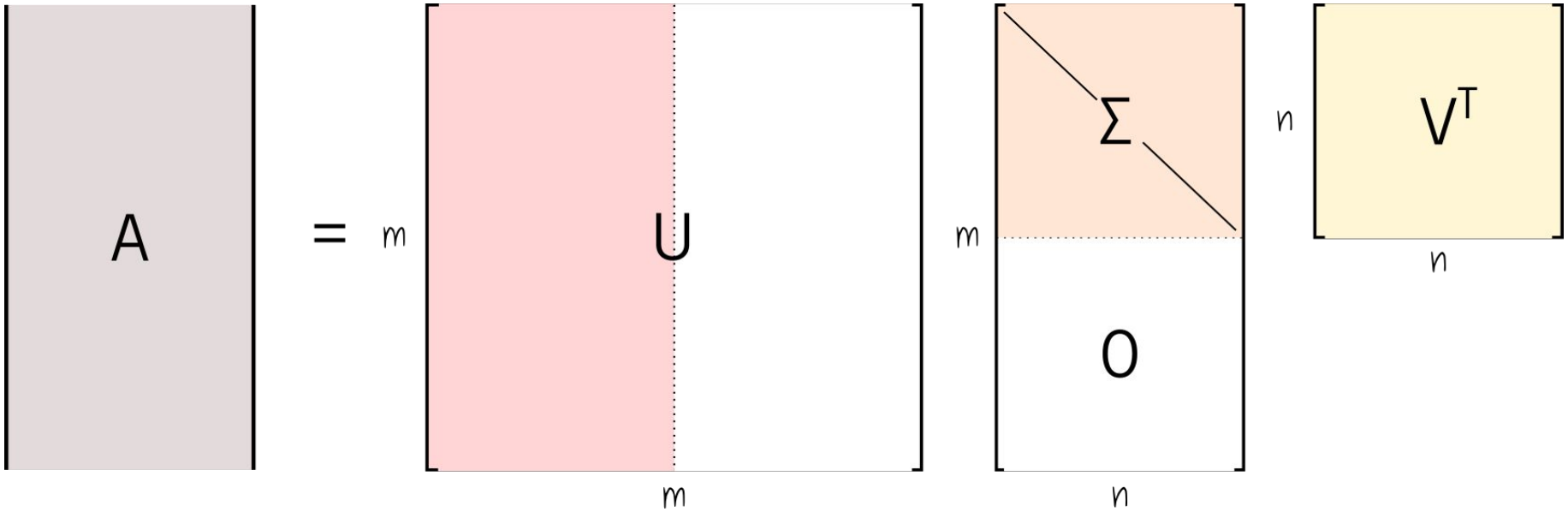
$$X = U\Sigma V^T$$

gdzie U i V to macierze ortogonalne, kształtu `U=(n,n)` oraz `V^T=(d,d)`

`Σ=(n,d)` to macierz przekątniowa z singular values zapisanymi nierosnąco

- **własności:** istnieje zawsze, liczby rzeczywiste, bardzo stabilne numerycznie

Singular Value Decomposition (SVD)



Rozwiązanie analityczne - SVD

- pseudoodwrotność: $X^{-1} \approx X^{\dagger} = V\Sigma^{+}U^{T}$

  gdzie $\Sigma+$ to odwrotności niezerowych wartości osobliwych + transpozycja
- **regresja liniowa:** $w = X^{\dagger}y$
- **zalety:**
  - szybkie - SVD ma doskonałe implementacje
  - precyzja - unikalne, dokładne rozwiązanie
  - stabilność numeryczna - nawet dla mocno skorelowanych cech

Rozwiązanie numeryczne

- MSE jest:
  - **różniczkowalne** - dużo optymalizatorów
  - **wypukłe (convex)** - jedno minimum globalne
- zadziała wszystko, np. spadek wzdłuż gradientu
- w praktyce używa się SVD, bo jest szybsze, stabilniejsze i nie ma hiperparametrów

Ridge & lasso regression

Regularyzacja w regresji liniowej

- **obserwacja:** gdy cecha ma bardzo dużą wagę, to niewielkie jej wahanie bardzo zmienia predykcję
- jest to **duża wariancja**, czyli po prostu **przeuczenie (overfitting)**
- **regularyzacja** zmniejsza wagi, potencjalnie do zera (selekcja cech)

- **wymaga** przeskalowanych zmiennych, bo wtedy wagi też są tej samej skali

Regularyzacja L2 oraz L1

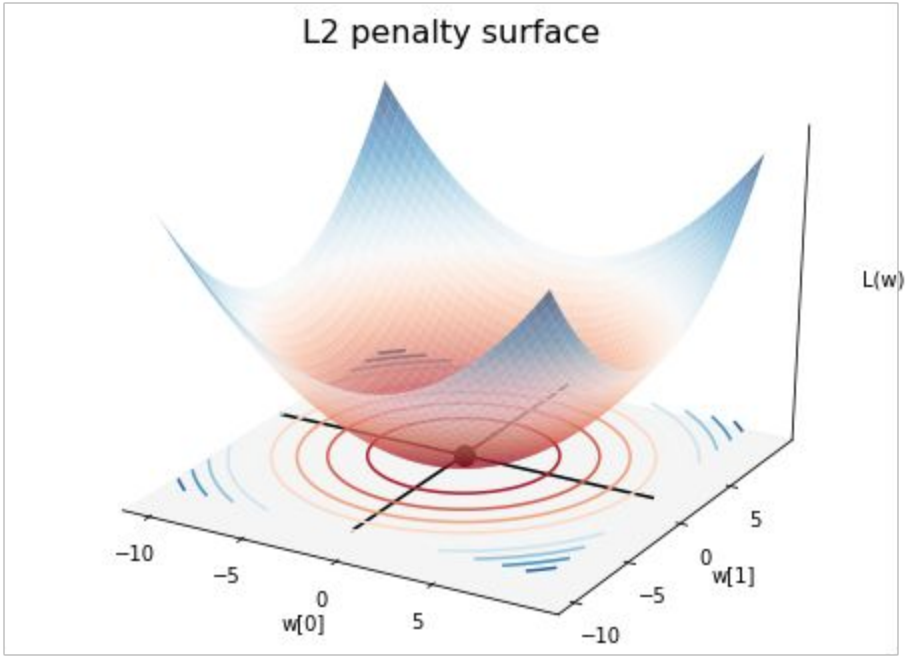
- **ridge regression** daje normę L2 wag do kosztu, a **LASSO regression** normę L1:

$$L_{ridge}(w) = \frac{1}{n}||y - Xw||^2 + \lambda||w||^2$$

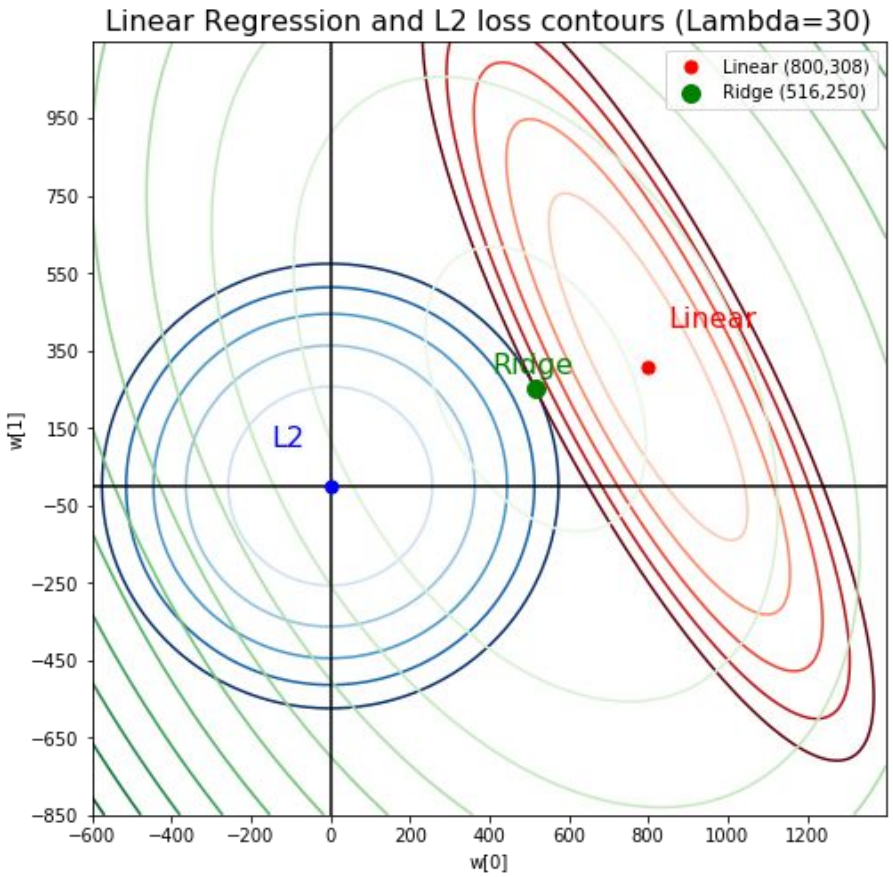
$$L_{LASSO}(w) = \frac{1}{n}||y - Xw||^2 + \lambda||w||_1$$

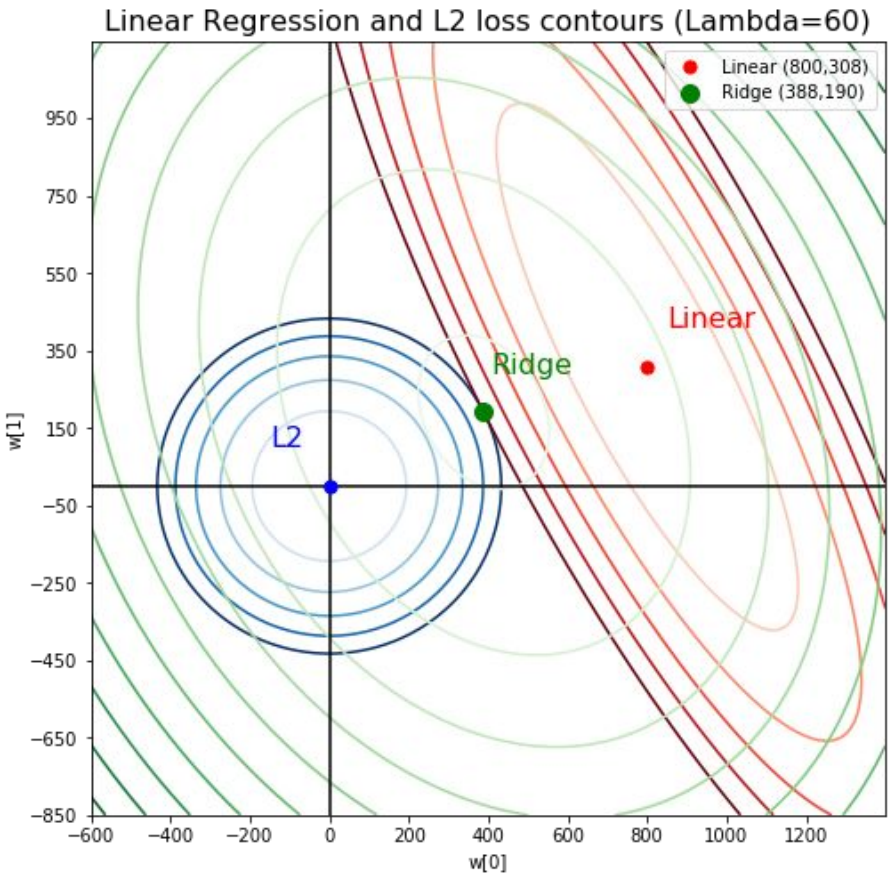
- L2 zmniejsza wagi i jest proste w obliczaniu (bo różniczkowalne)
- L1 często daje wagi równe zero - **embedded feature selection** (realizowana "w środku" modelu)

Regularyzacja L2 - wizualizacja

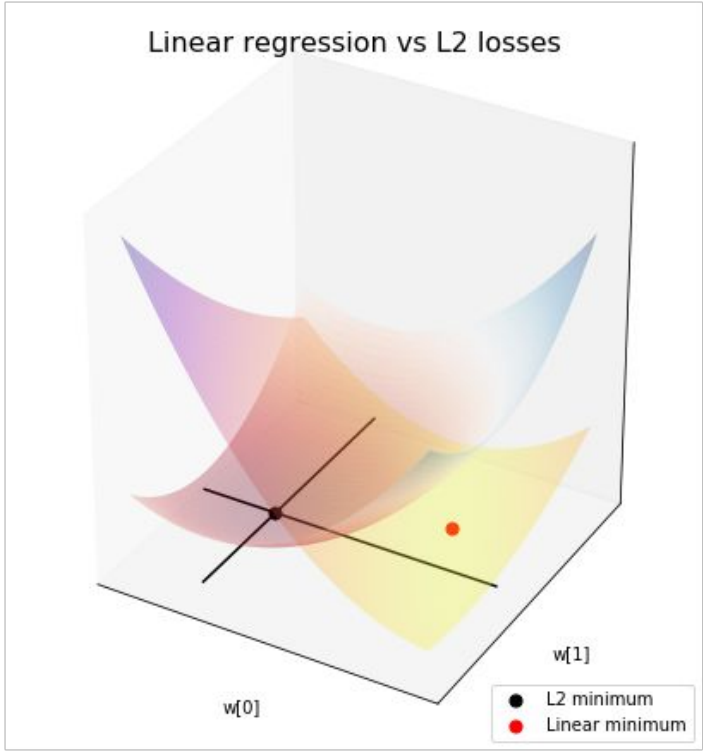


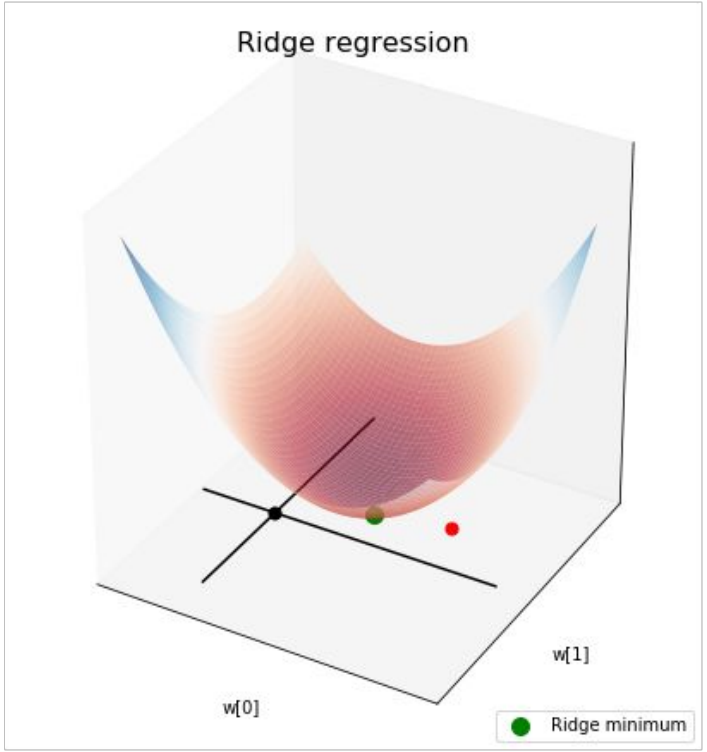
Regularyzacja L2 - wizualizacja



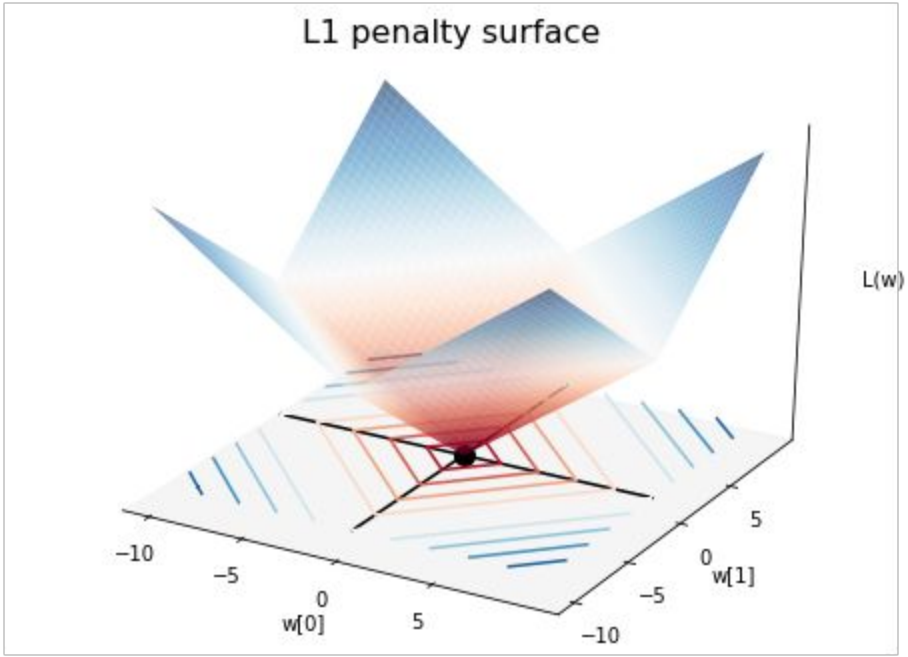


Regularyzacja L2 - wizualizacja

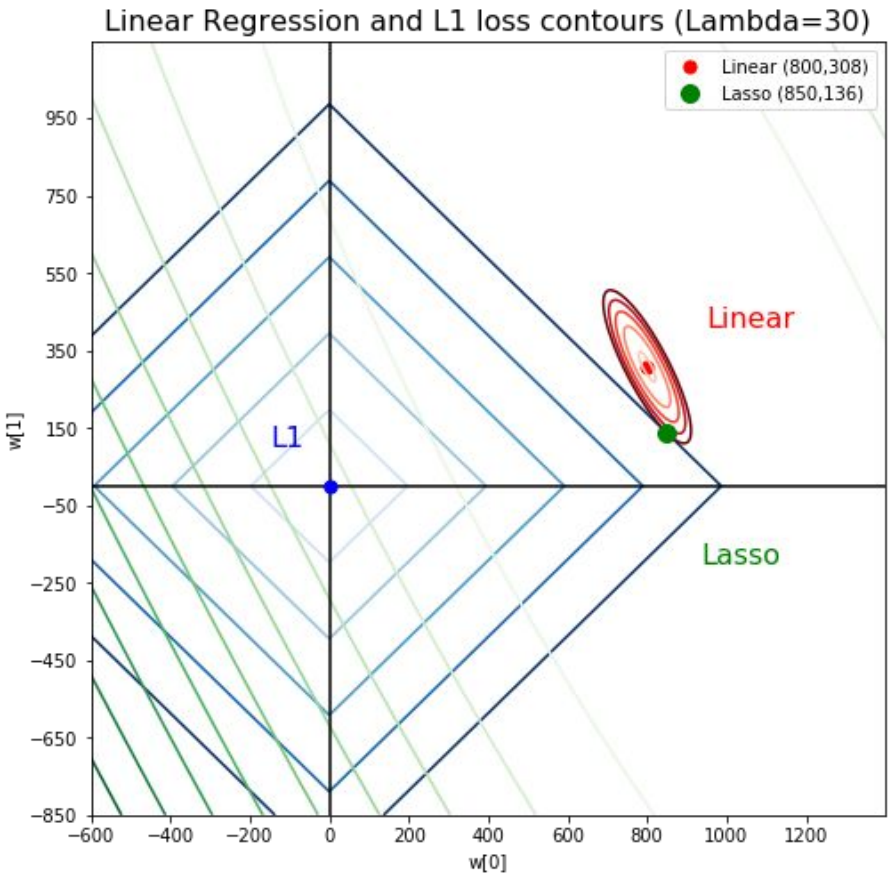


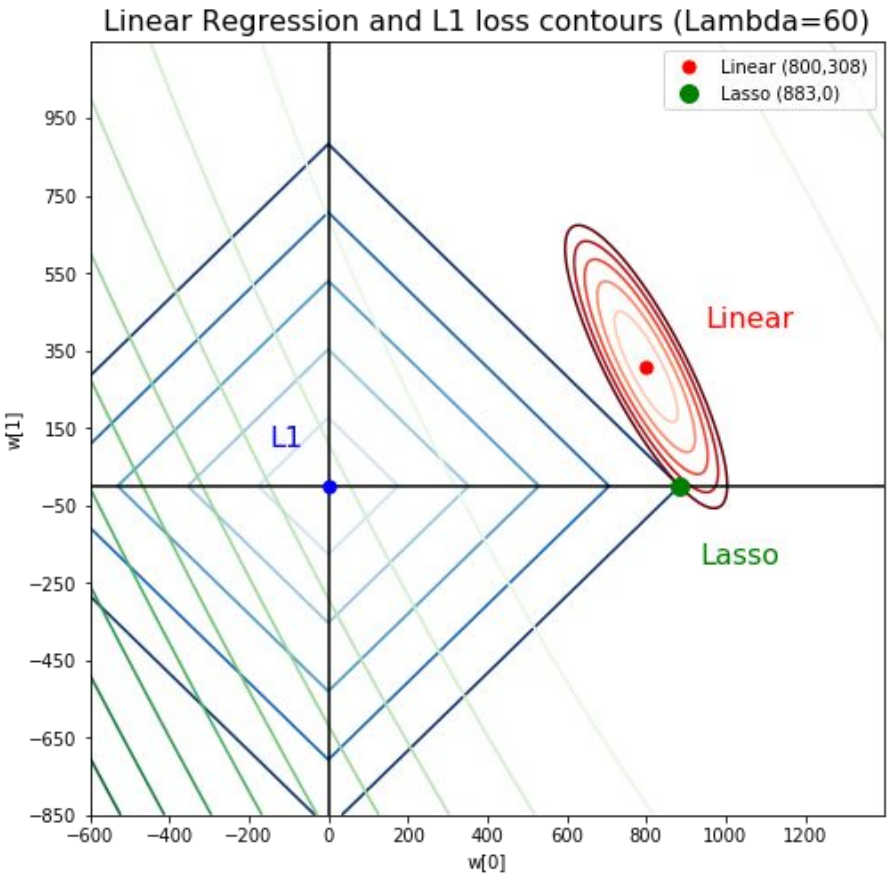


Regularyzacja L1 - wizualizacja

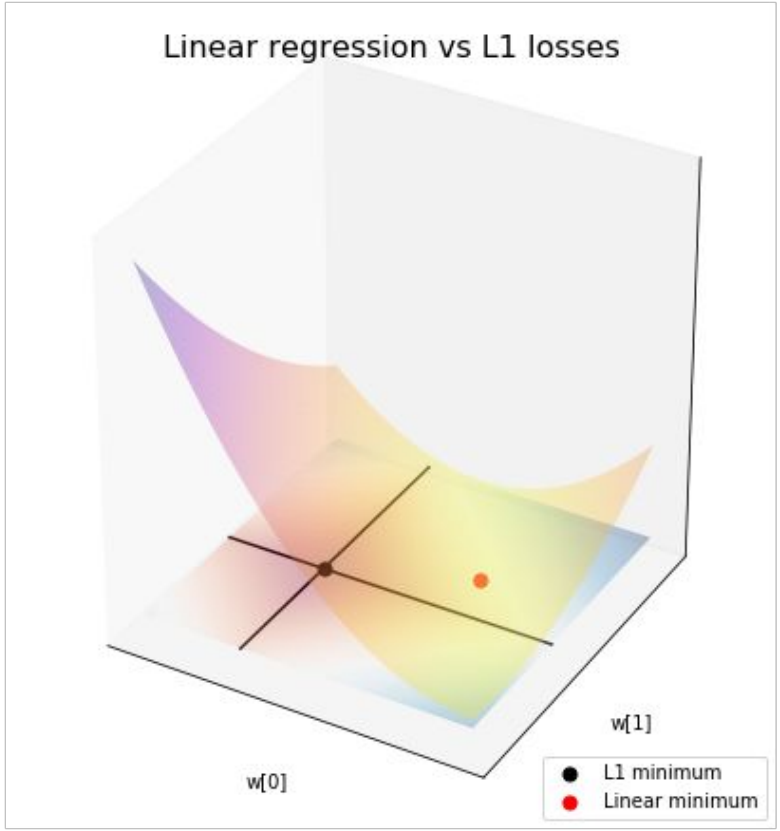


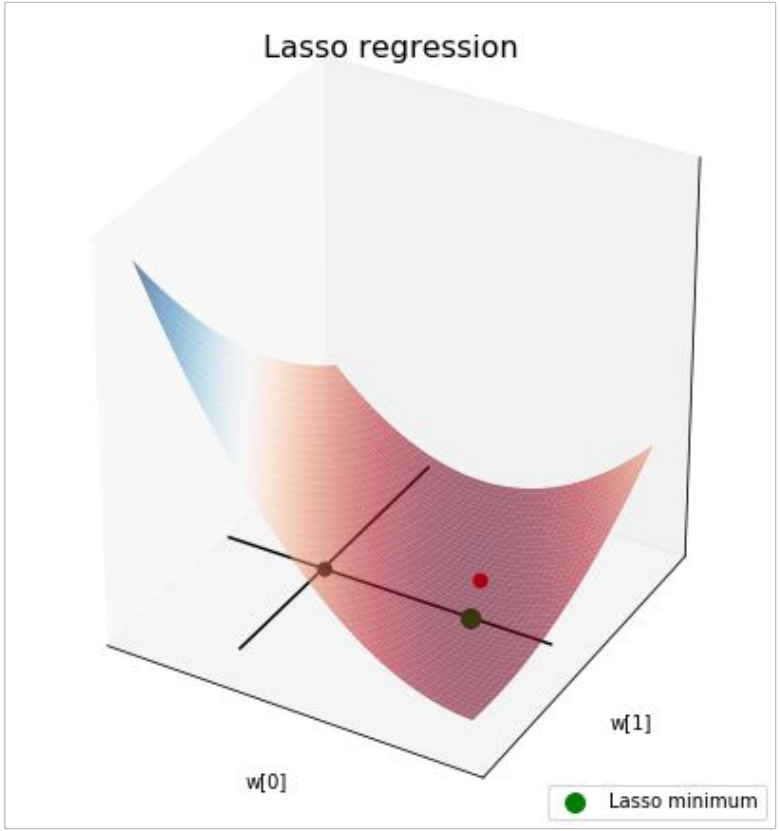
Regularyzacja L1 - wizualizacja





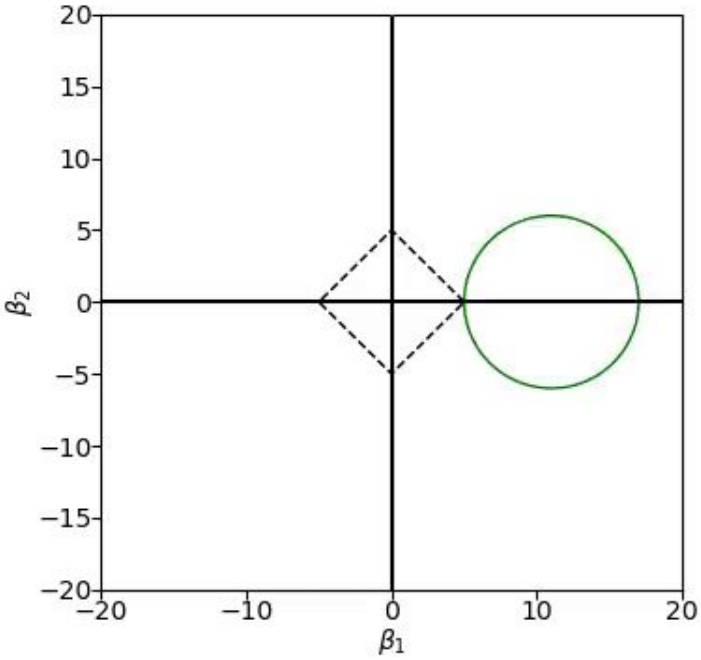
Regularyzacja L1 - wizualizacja

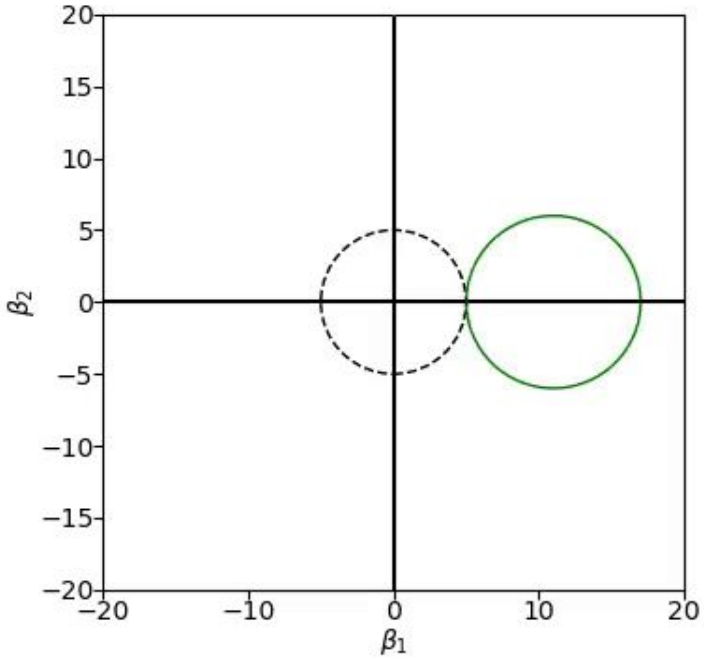




Regularyzacja L1 vs L2 - wizualizacja

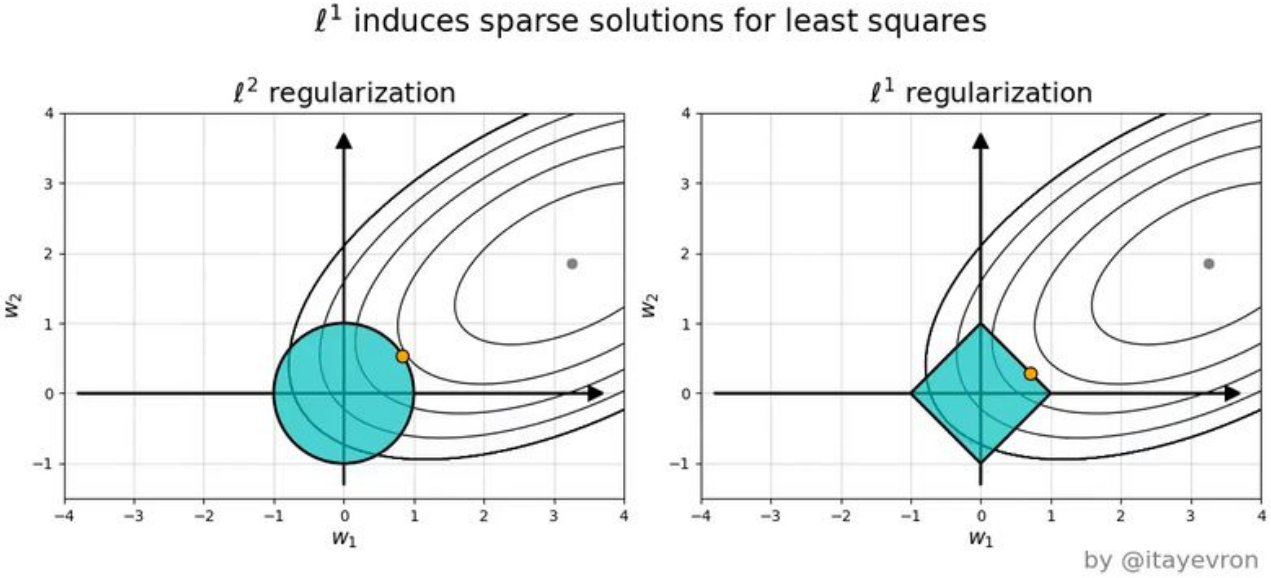
- zielony - jedna z cech ma wagę zero
- niebieski - wszystkie cechy niezerowe





Regularyzacja L1 vs L2 - wizualizacja

- dla różnych problemów optymalizacji norma L1 czesto prowadzi do wag na osiach układu współrzędnych, z wagą cechy zero



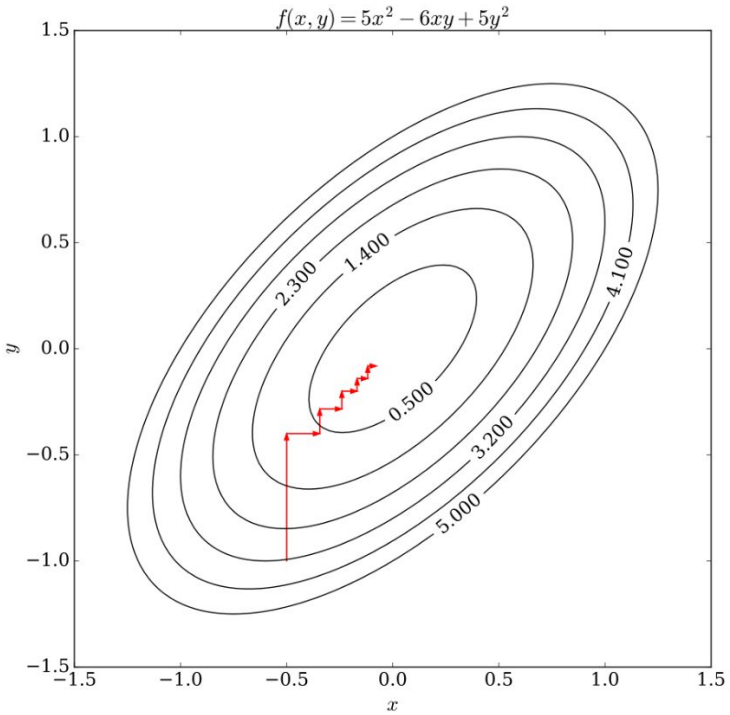
Ridge regression (L2) - obliczanie

- mamy do wyboru wiele różnych solwerów - wynik ten sam, ale różnią się szybkością w różnych sytuacjach
- **SVD** - pozwala bardzo wydajnie zrobić **LOOCV (Leave-One-Out Cross-Validation)** i dobrać siłę regularyzacji (`RidgeCV` w Scikit-learn)
- **Cholesky decomposition** - dane gęste, nieco szybsze i mniej stabilne niż SVD
- **conjugate gradient** - dane rzadkie
- **L-BFGS** - kiedy chcemy wymusić tylko pozytywne wagi

LASSO regression (L1) - obliczanie

- **nieróżniczkowalne** - typowe solwery nie działają
- używamy **coordinate descent (CD)** - typowy solwer dla problemów nieróżniczkowalnych
- zbiega do minimum lokalnego, więc dobra dla **funkcji wypukłych (convex)**
- **idea:**
  - idź po kolei po kolejnych zmiennych (albo losowo)
  - dla aktualnej zmiennej dokonaj minimalizacji, np. przeszukiwaniem liniowym (line search); pozostałe są w tej chwili stałe
  - powtarzaj aż do osiągnięcia zbieżności

Coordinate descent - wizualizacja



Regularization path

- CD pozwala na **efektywny** dobór siły regularyzacji λ
- tzw. **regularization path:**
  - sprawdzamy wartości λ rosnąco, dla każdej robimy np. 5-fold CV
  - wagi z poprzedniej iteracji służą jako tzw. **warm start**
  - bardzo przyspiesza zbieżność
  - `LassoCV` w Scikit-learn

Regularyzacja ElasticNet

- **ElasticNet** to jednocześnie regularyzacja L2 oraz L1:

$$L(w) = \frac{1}{N}||y - Xw||^2 + \lambda||w||_2^2 + \alpha||w||_1$$

- optymalizacja jak LASSO (coordinate descent, regularization path)
- **więcej hiperparametrów**, siatka jest kwadratowa zamiast liniowej

Złożoność obliczeniowa

- **SVD:** $O(nd^2)$ (Golub, Van Loan "Matrix Computations, 4th ed.", str. 493)
- **Cholesky:** $O(nd^2)$, ale mniejsza stała (źródło)
- **inne solwery numeryczne:**
  - nie da się stwierdzić w ogólnym wypadku, zależy od liczby iteracji
  - w praktyce bardzo szybkie, szczególnie jak dobierzemy odpowiedni solwer

Regresja liniowa - zalety i wady

Zalety:

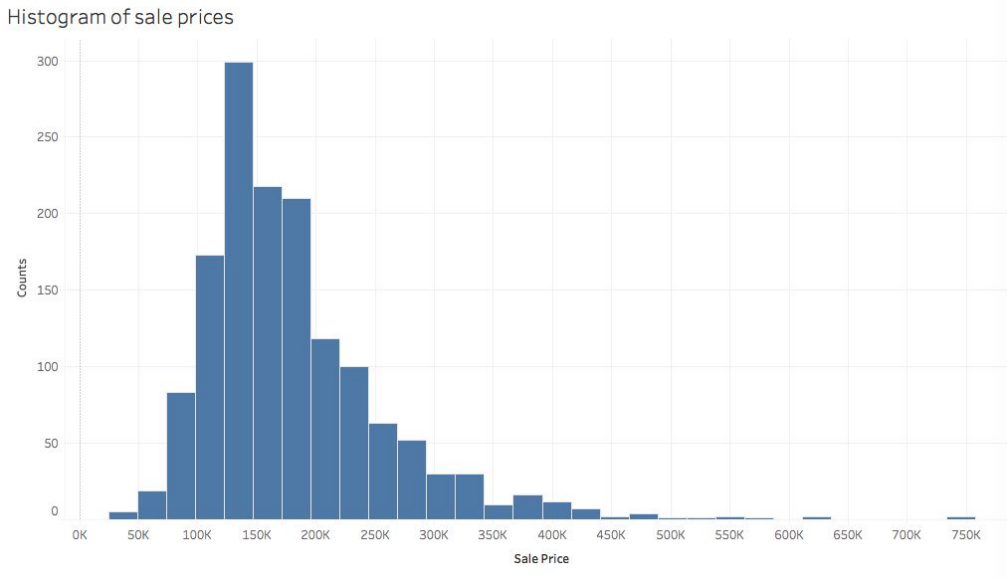
- prostota
- szybkość i skalowalność
- regularyzacja L1, L2 i ElasticNet
- mało hiperparametrów
- dobry baseline

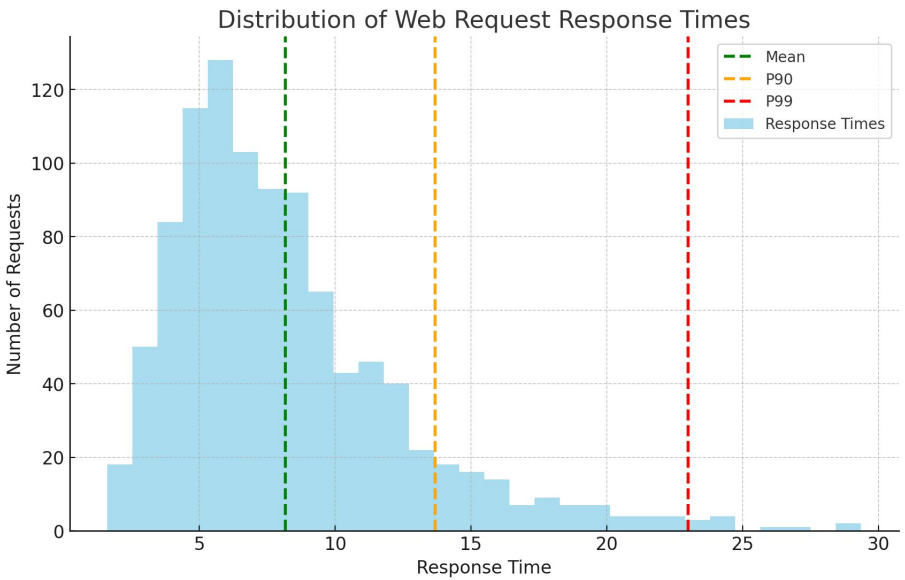
Wady:

- relatywnie czułe na outliery
- duża czułość na dobór hiperparametrów
- w szczególności nie działa dobrze dla regresji niezbalansowanej

Regresja kwantylowa

Regresja niezbalansowana





Problem z regresją liniową

- estymuje **conditional mean**, czułe na outliery
- jeżeli zmienna zależna nie ma rozkładu normalnego, to działa **bardzo słabo**
- jest wiele algorytmów tzw. **robust regression** - zwraca mniejszą (lub żadną) uwagę na outliery
- np. estymujemy warunkową medianę zamiast średniej

Least Absolute Deviations (LAD) regression

$$L(w) = MAE(w) = \frac{1}{n}|y_i - \hat{y}_i| = \frac{1}{n}||y - Xw||_1$$

- **funkcja kosztu:** mean absolute error (MAE)
- **algorytm:** programowanie liniowe
- estymuje medianę warunkową
- **zalety:** odporne na outliery, wprost optymalizuje MAE
- **wady:** dużo bardziej kosztowne obliczeniowo, tylko regularyzacja L1

Regresja kwantylowa (quantile regression)

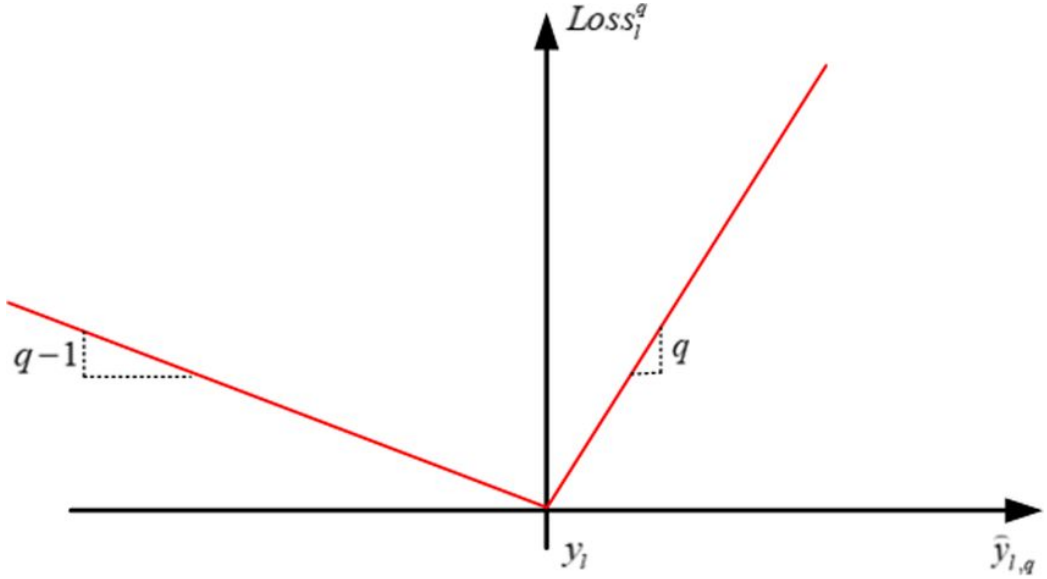
- **funkcja kosztu:** pinball loss (quantile loss)
- **algorytm:** programowanie liniowe
- estymuje warunkowy kwantyl q
- LAD to szczególny przypadek
- **zalety:** elastyczna estymacja
- **wady:** koszt obliczeniowy

$$r = y_i - x_i^T w$$

$$PB_q(r) = \begin{cases} (1-q) * |r| & r < 0 \\ 0 & r = 0 \\ q * |r| & r > 0 \end{cases}$$

Pinball loss (quantile loss)

$$PB_q(r) = q\max(y - \hat{y}, 0) + (1 - q)\max(\hat{y} - y, 0)$$



Quantile regression - zastosowania

- **robust regression** - mediana jest bardziej odporna na outliery
- **estymacja ogona rozkładu:**
  - czasem interesuje nas gwarancja "worst experience", czyli kwantyle 0.9 czy 0.99
  - *What is the website p99 latency?*
  - *For our package delivery SLA, what will be the guaranteed delivery time for 90% of packages?*
- dodatkowe przykłady

Quantile regression - obliczanie

- CD by zadziałał, ale da się lepiej i szybciej
- **programowanie liniowe (linear programming):**
  - problem optymalizacji liniowego wyrażenia
  - z liniowymi ograniczeniami
- **powszechne** w logistyce, optymalizacji produkcji itp.
- **zalety:** wygodne w zapisie, istnieją **bardzo** wydajne algorytmy

Find a vector	$\mathbf{x}$
that maximizes	$\mathbf{c}^\mathsf{T}\mathbf{x}$
subject to	$A\mathbf{x} \leq \mathbf{b}$
and	$\mathbf{x} \geq \mathbf{0}.$

LAD regression - obliczanie

$$\underset{w}{\text{minimize}} \quad \sum_{i=1}^{n} \left| y_i - x_i^T w \right|$$

- **zadanie:** zminimalizować wartości bezwzględne rezyduów
- **zmienne:** wektor wag w

- wartość bezwzględna jest nieliniowa - trzeba to przekształcić

LAD regression - obliczanie

- zmienne oraz ograniczenia pomocnicze
- tylko wartość bezwzględna spełnia te ograniczenia
- u wyjdą zawsze nieujemne
- [wyprowadzenie]
- [wyjaśnienie]
- [alternatywne sformułowania]

$$\underset{w}{\text{minimize}} \quad \sum_{i=1}^{n} \left| y_i - x_i^T w \right|$$

$$\downarrow$$

$$\begin{aligned} \underset{w}{\text{minimize}} \quad & \sum_{i=1}^{n} u_i \\ \text{subject to} \quad & u_i \geq y_i - x_i^T w \qquad \text{for } i = 1, \ldots, n, \\ & u_i \geq -(y_i - x_i^T w) \quad \text{for } i = 1, \ldots, n \end{aligned}$$

Quantile regression - obliczanie

- bardzo podobnie, tylko z odpowiednimi wagami dla kwantyla
- sformułowanie

- można też do obu sformułowań (LAD i quantile) dodać regularyzację L1
- L2 nie działa, bo ma kwadrat, więc jest nieliniowe

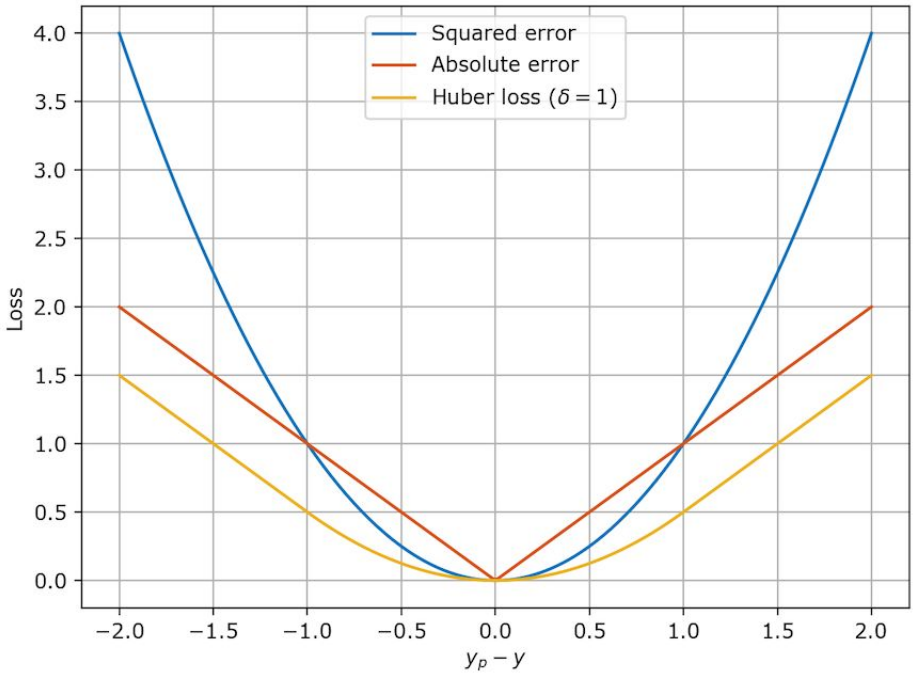
Huber regression

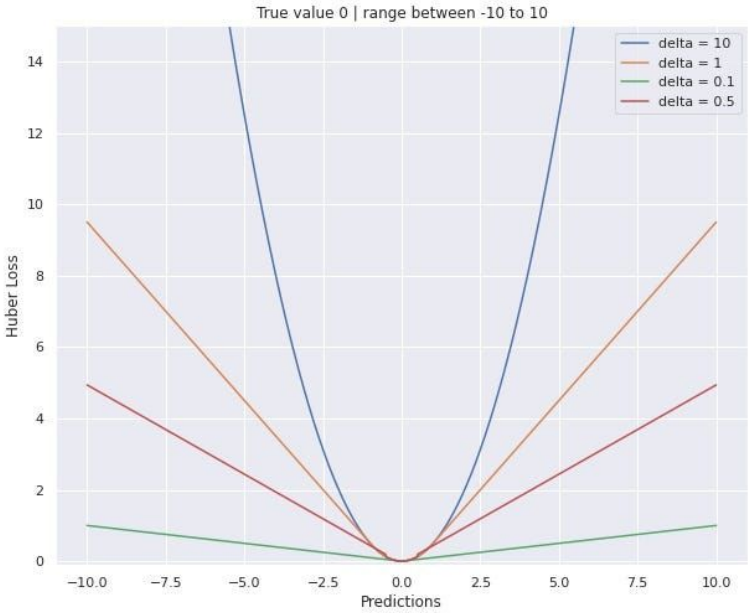
Huber regression

$$L_{huber}(w) = \begin{cases} \frac{1}{2}||y - Xw||^2 & \text{for } |y - Xw| \leq \epsilon \\ \epsilon * (||y - Xw||_1 - \frac{1}{2}\epsilon) & \text{otherwise} \end{cases}$$

- **funkcja kosztu:** Huber loss
- **algorytm:** L-BFGS
- MSE blisko zera, MAE dalej
- odporne na outliery, a prawie wszędzie gładkie i różniczkowalne
- **zalety:** łączy zalety MSE i MAE, szybkie, mało czułe na ε
- **wady:** tylko regularyzacja L2

Huber loss


Squared error
Absolute error
Huber loss ($\delta = 1$)
Loss
$y_p - y$
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


True value 0 | range between -10 to 10
delta = 10
delta = 1
delta = 0.1
delta = 0.5
Huber Loss
Predictions
14
12
10
8
6
4
2
0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

Huber regression - obliczanie

- minimalizujemy funkcję kosztu - trzeba **solwera numerycznego**
- **własności Huber loss:**
  - różniczkowalny
  - ciągły
  - wypukły
- mamy szeroki wybór optymalizatorów

Metody optymalizacji

- **zero-order (bezgradientowe):**
  - funkcje nieciągłe, nieróżniczkowalne etc.
  - np. simplex method, genetic algorithms
- **first-order (gradientowe):**
  - dla funkcji różniczkowalnych, bardzo skalowalne
  - np. gradient descent
- **second-order (hesjanowe):**
  - używają informacji o drugiej pochodnej, bardzo szybka zbieżność
  - np. metoda Newtona, L-BFGS

Second-order optimization

- używamy informacji o **krzywiźnie (curvature)** funkcji
- **dokładniejsze oszacowanie** funkcji - rozwijamy szereg Taylora do 2 elementu (wartość, gradient, hesjan)

$$f(x_k + t) \approx f(x_k) + f'(x_k)t + \frac{1}{2}f''(x_k)t^2$$

- są to **metody Newtona (Newton / Newton-Raphson methods)**
- **zalety:** bardzo szybka zbieżność
- **wady:** nieskalowalne dla wielu zmiennych

Metoda Newtona

- 1-wymiarowa:

$$x_{k+1} = x_k + t = x_k - \frac{f'(x_k)}{f''(x_k)}$$

- wielowymiarowa:

$$x_{k+1} = x_k - [f''(x_k)]^{-1} f'(x_k)$$

- gradient i hesjan:

$$f'(x) = \nabla f(x) = g_f(x) \in \mathbb{R}^d$$

$$f''(x) = \nabla^2 f(x) = H_f(x) \in \mathbb{R}^{d \times d}$$

- **problem:** to jest drogie, trzeba obliczyć i odwrócić hesjan w każdej iteracji!
- **złożoność**: $O(d^3)$

Quasi-Newton methods

- **quasi-Newton methods:** używamy przybliżenia hesjanu, używając *f(x)* oraz *f'(x)*
- uogólnienie **metody siecznych** na wiele wymiarów
- absolutnie najpopularniejszą metodą jest **L-BFGS**

  (Limited memory Broyden–Fletcher–Goldfarb–Shanno algorithm)
- szczegóły: link 1, link 2, link 3, link 4, link 5, link 6, link 7
- **zalety:** znacznie mniejsza złożoność obliczeniowa i pamięci $O(d^2)$ lub mniej
- **wady:** wolniejsza zbieżność, czasem nie działa dla "trudnych" funkcji

Podsumowanie

Modele liniowe

$$\hat{y} = w_0 1 + w_1 x_1 + \ldots + w_d x_d = x^T w$$

Nazwa	Funkcja kosztu	Algorytm (optymalizator)
Linear regression	MSE	SVD
Ridge reg.	MSE + L2	SVD i inne
LASSO reg.	MSE + L1	Coordinate Descent (CD)
ElasticNet reg.	MSE + L2 + L1	CD
Quantile reg., LAD	Pinball loss, MAE	Programowanie liniowe
Huber reg.	Huber loss	L-BFGS